



A Comparative Study of Container Hardening Strategies



Authors: Amit Soni; Dr. Myungah (Grace) Park, Ph.D.

Department of Computer Science, University of Central Oklahoma, Edmond, Oklahoma

1. Abstract

Docker containers depend on base images to provide operating system files and runtime dependencies. The official python:latest image, built on full Debian, serves as the industry standard due to stability and broad compatibility. However, this general-purpose design includes unnecessary tools (compilers, utilities, system libraries) which increase the attack surface and expose applications to avoidable vulnerabilities.

This study evaluates container hardening: eliminating unused software to mitigate risk without compromising functionality. On January 7, 2026, we containerized a Python web application using four strategies: Baseline (Debian), Debian Slim, Alpine Linux (lightweight OS with musl libraries), and Distroless (removes OS shell).

Results: Alpine achieved 99.9% vulnerability reduction (1 CVE vs 1,384 in Baseline) and 92% size reduction (125 MB vs 1.65 GB). Notably, Distroless still harbored 203 CVEs including 4 Critical (removing the shell is insufficient if core libraries remain unchanged). Alpine delivers optimal balance of security, supply chain simplicity, and storage efficiency.

2. Introduction: The Hidden Danger

Developers use default base images (like python:latest) for convenience but unknowingly inherit massive security risks.

The Problem: Default images are "black boxes", entire operating systems with compilers and thousands of unused files.

The Risk: Larger attack surface means more unpatched vulnerabilities (CVEs) for exploitation.

The Objective: Scientifically measure if hardening truly reduces risk or is just marketing hype.

3. Methodology (Jan 7, 2026)

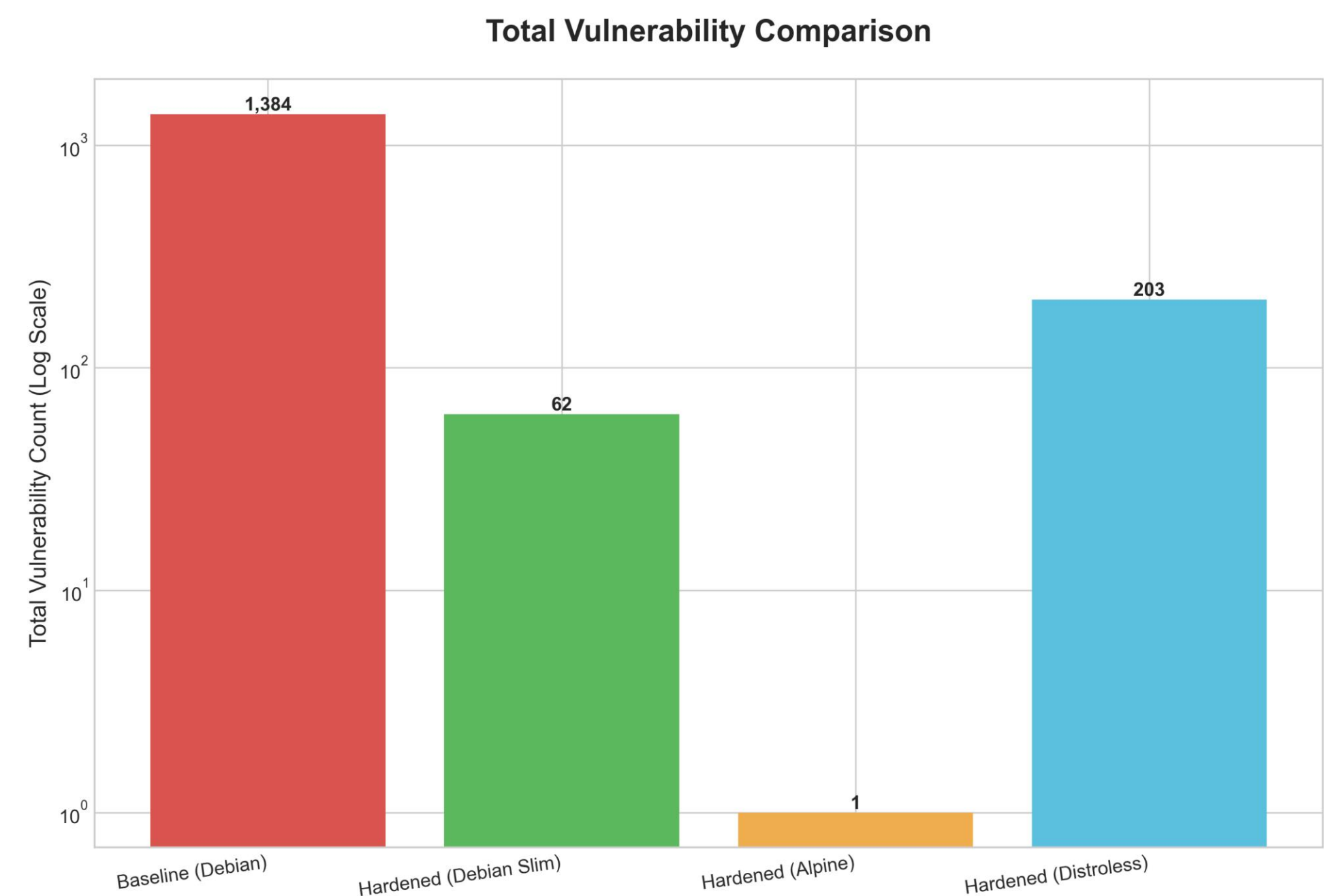
Controlled experiment: Fixed Python web application across four build strategies.

- ❖ **Baseline:** Default Debian OS (Control)
- ❖ **Debian Slim:** Removed documentation/manuals
- ❖ **Alpine Linux:** Minimalist OS (musl library)
- ❖ **Distroless:** Removed command shell (/bin/sh)

Toolkit:

- ❖ **Trivy & Gype** - CVE scanning
- ❖ **Syft** - Software component inventory
- ❖ **Dive** - Filesystem wasted space analysis

4. Vulnerability Reduction (Log Scale)

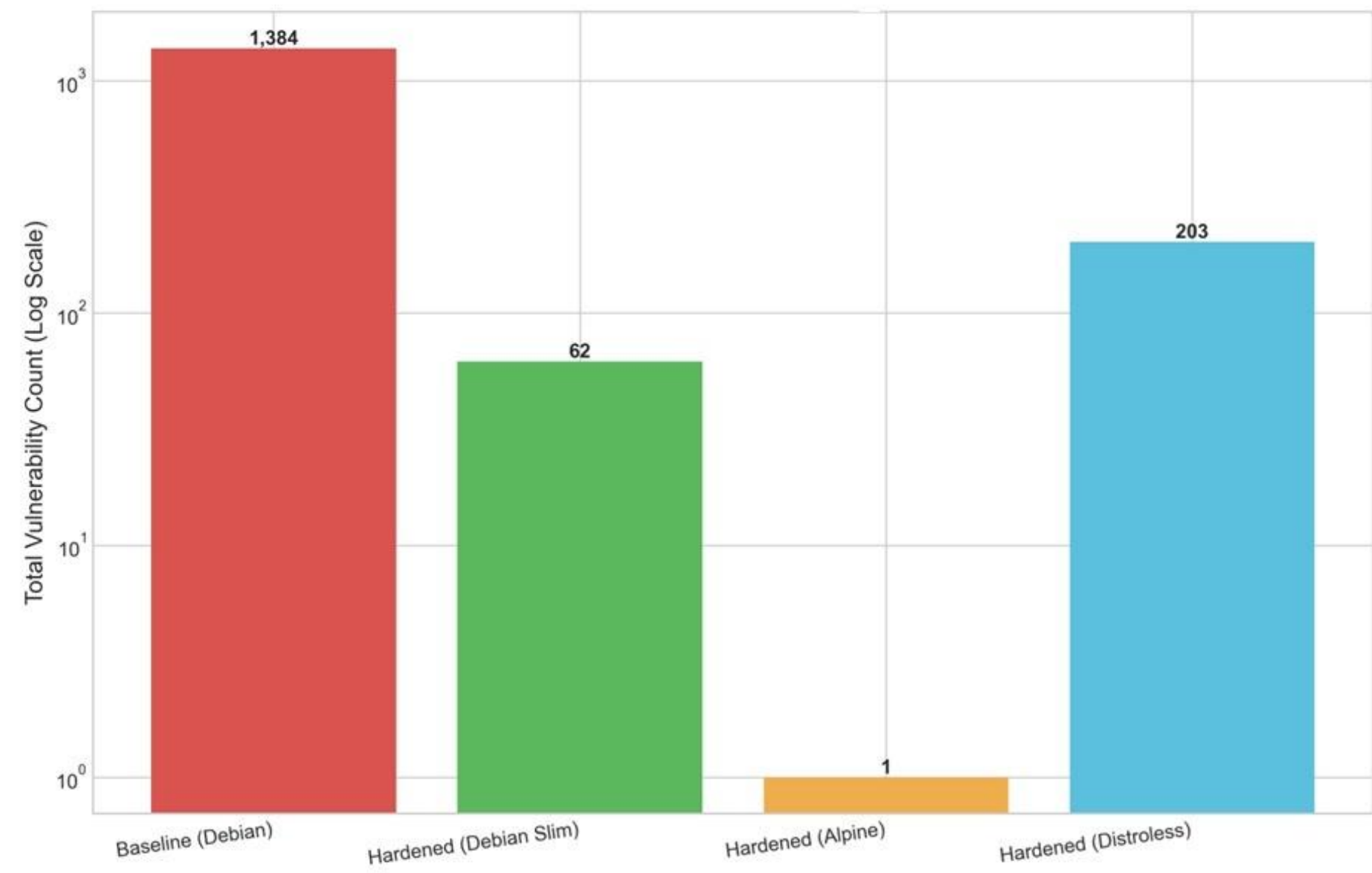


Finding: Alpine reduces vulnerabilities by 99.9%.

Distroless has 203 CVEs despite removing the shell.

The logarithmic scale demonstrates orders of magnitude improvement.

5. The "Distroless Anomaly"



Finding:

Expected:

✓ No shell = Maximum security

✓ Minimal attack surface

Actual:

✗ 203 CVEs (vs 62 Slim)

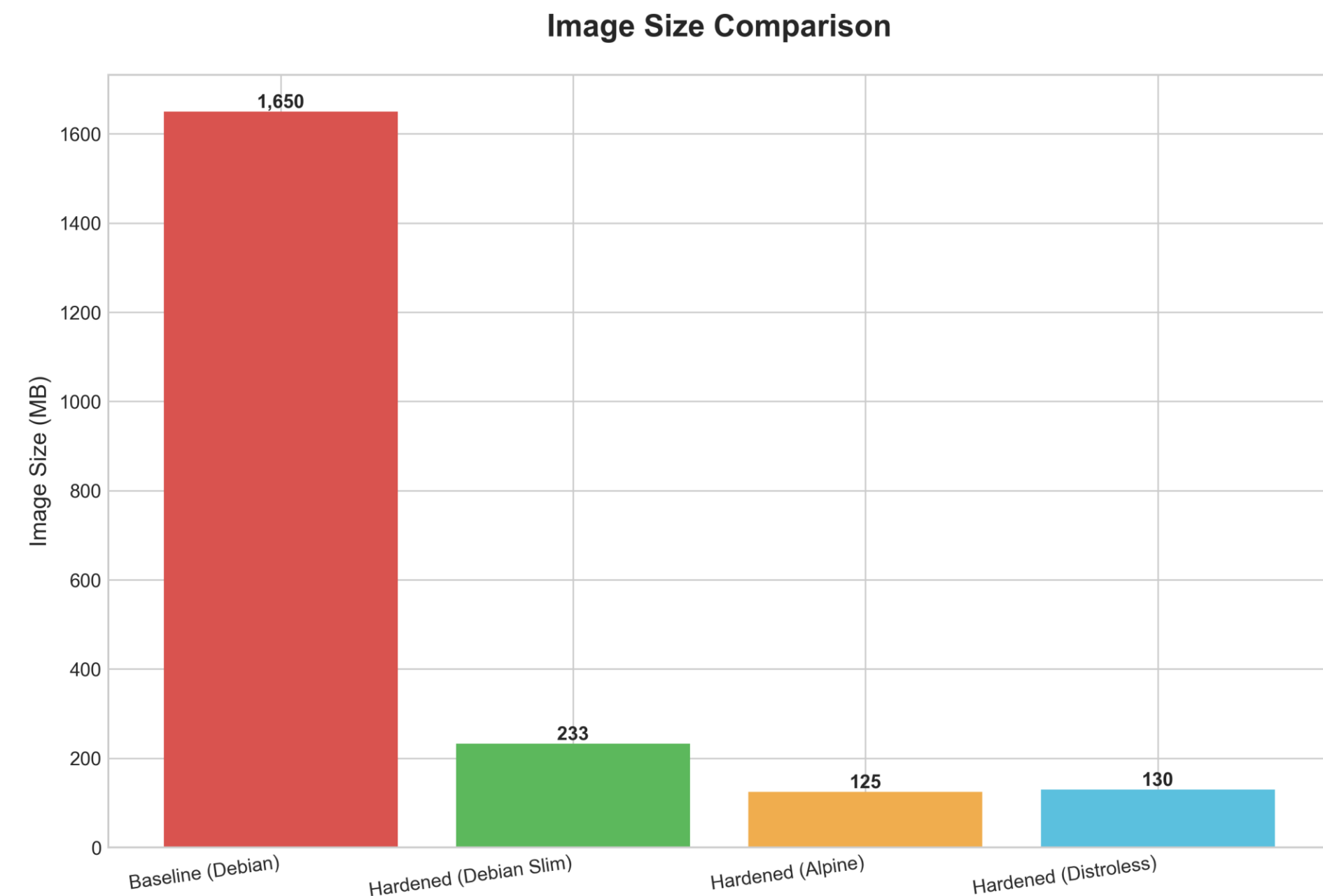
✗ 4 CRITICAL vulnerabilities

Root Cause:

Distroless removes the shell but retains Debian libraries (glibc).

Ecosystem matters more than component removal.

6. Image Size Efficiency



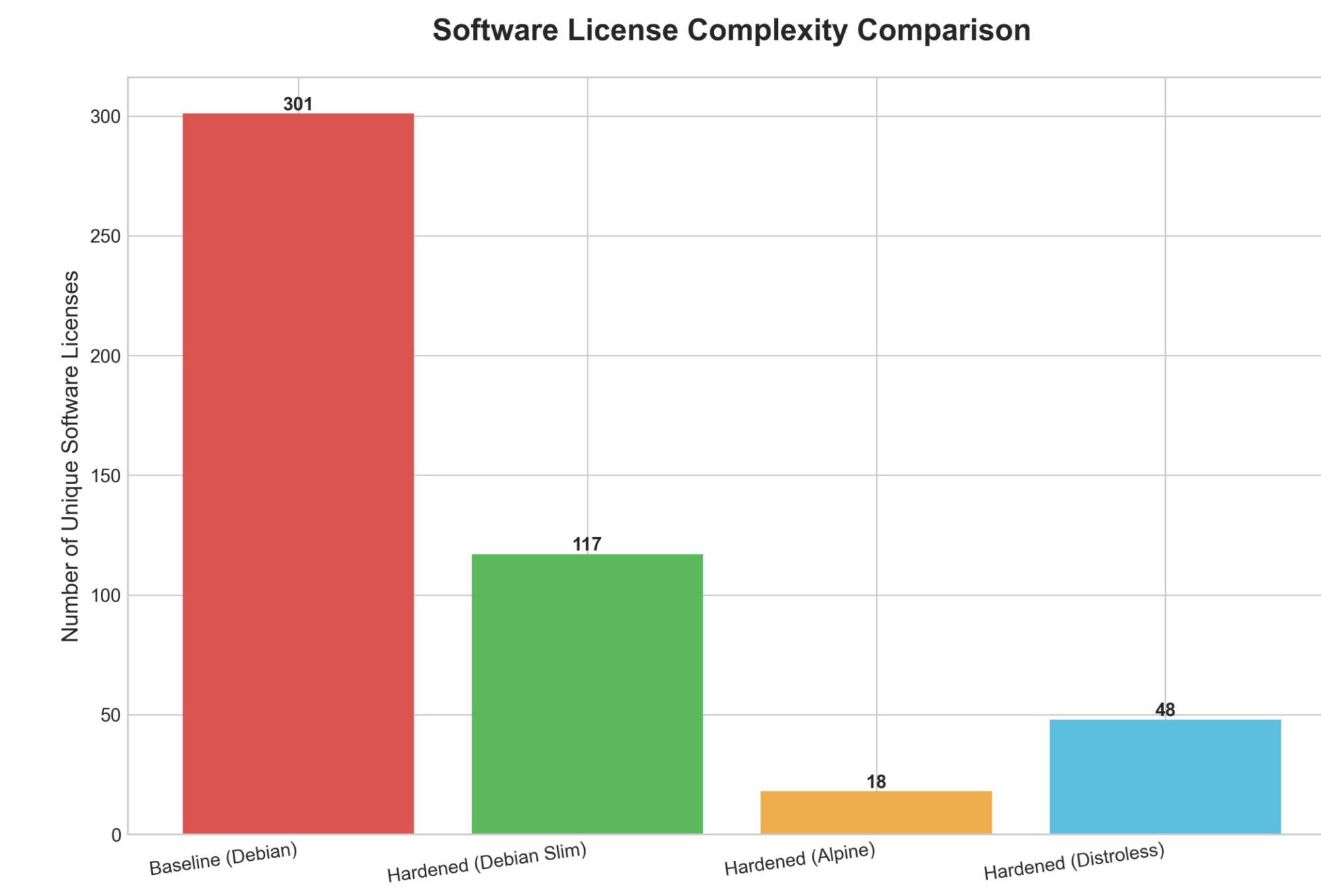
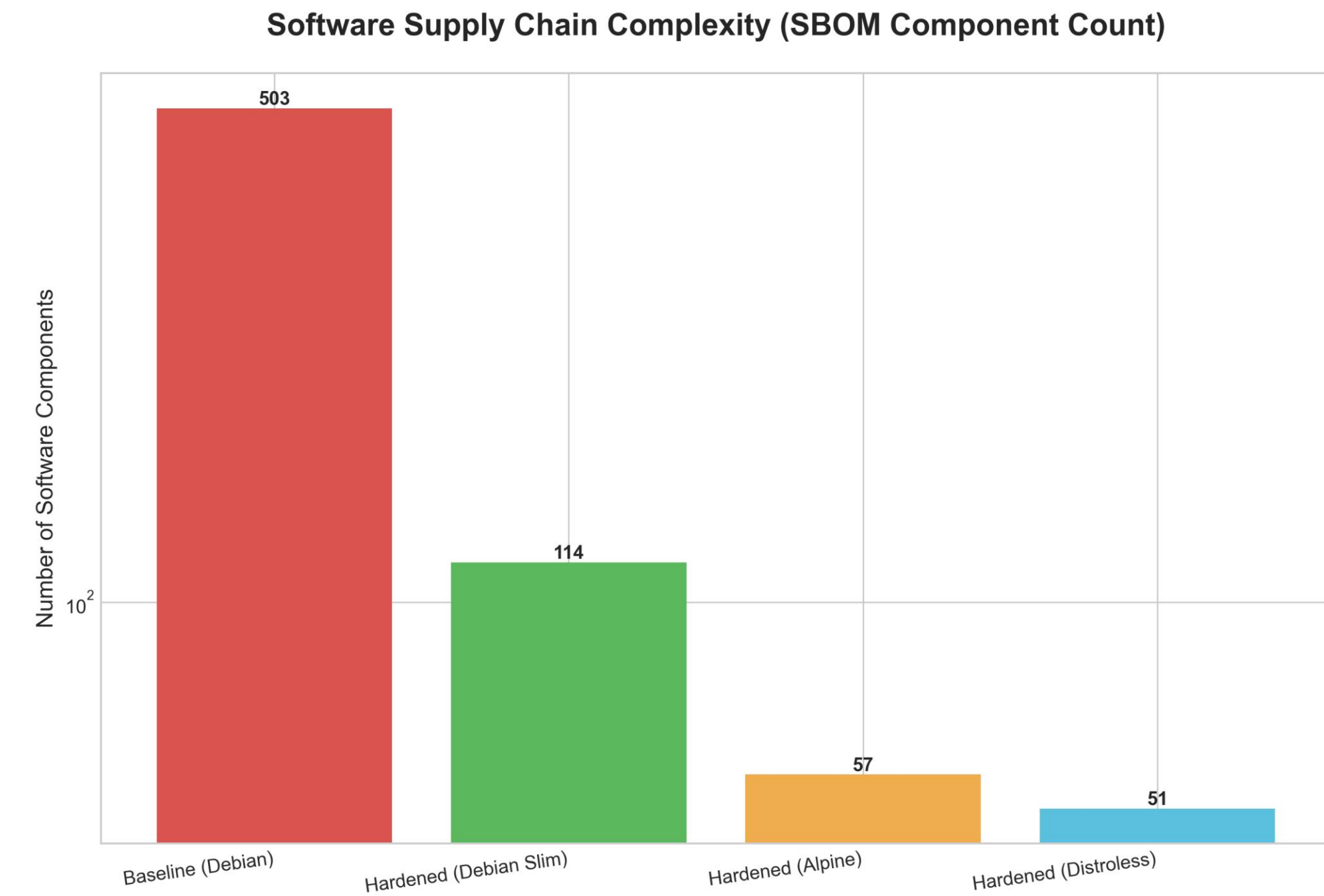
Finding: Alpine achieves smallest footprint (125 MB) with superior security.

7. Weakness Type Analysis (CWE)



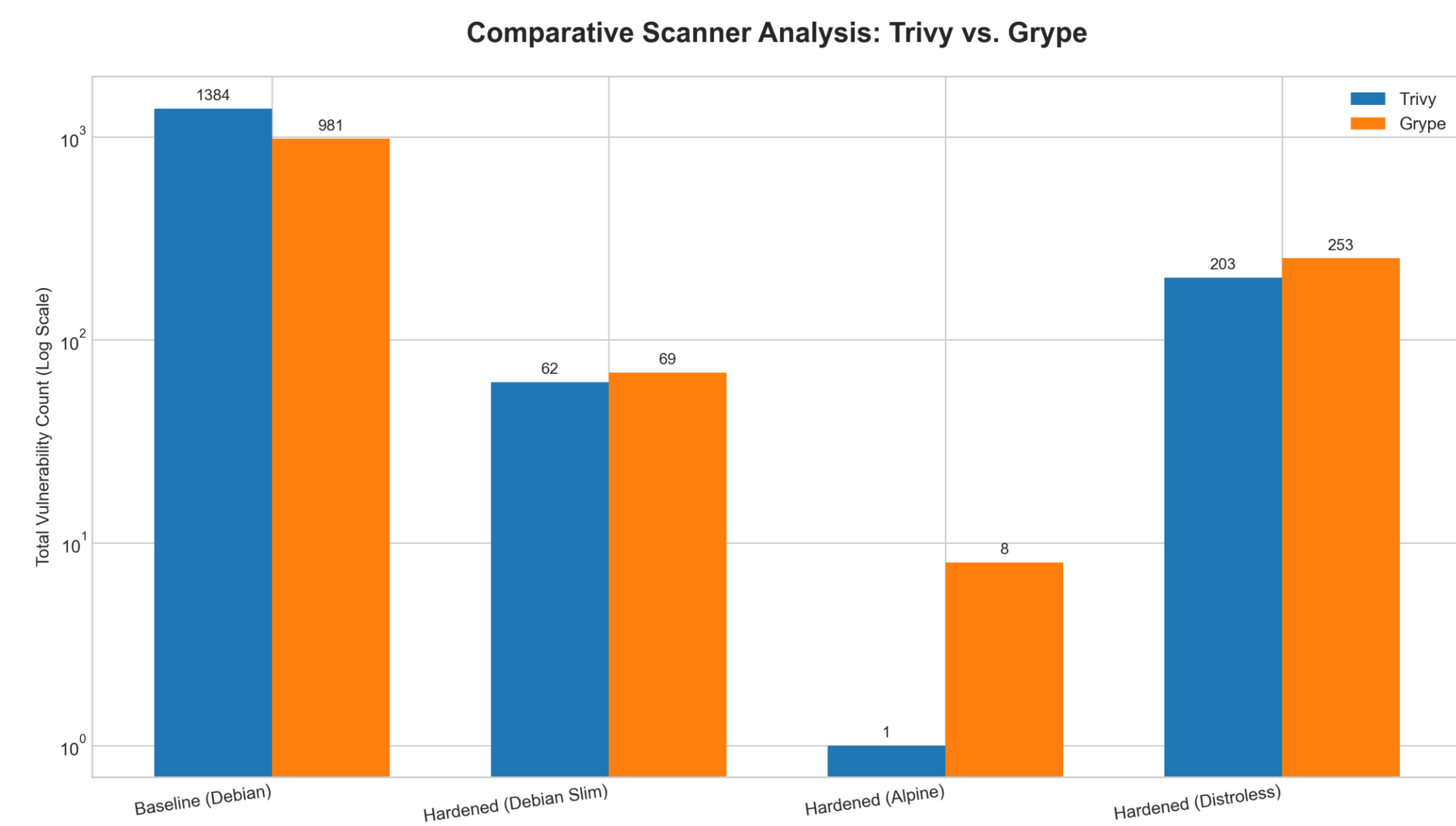
Finding: Alpine eliminates buffer errors (CWE-119) entirely. Distroless retains these due to glibc.

8. Software Supply Chain Simplification & License Complexity



Finding: Alpine reduces components by 89% (503 → 57) and licenses by 94% (301 → 18).

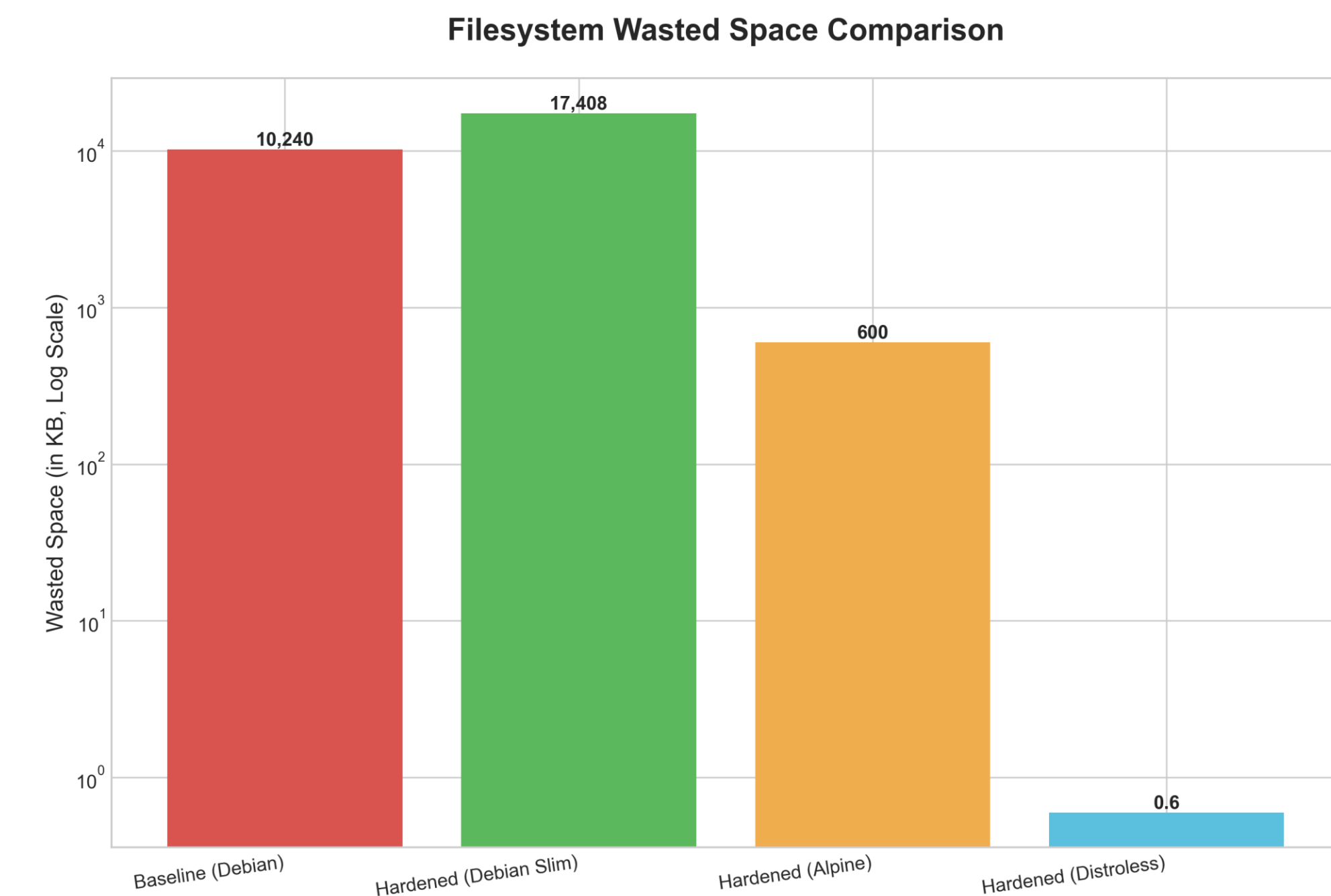
9. Multi-Scanner Validation



Finding: Trivy and Gype confirm Alpine is most secure. High correlation validates results.

Notable: The Alpine Anomaly Gype reports 8 CVEs vs Trivy's 1 (attributed to differences in database maintenance for musl-based systems)

10. Filesystem Efficiency



Finding: Distroless: 99.99% efficient. But efficiency ≠ security (still 203 CVEs).

11. Shared Vulnerability Analysis

Finding: Debian Slim and Distroless share 54 CVEs, confirming their common Debian/glibc lineage. Alpine shares only 1 CVE with either - switching to Alpine is a fundamental ecosystem shift, not just package reduction.

12. Conclusion

✓ **Alpine: Optimal Balance**

- ❖ 99.9% vulnerability reduction
- ❖ 92% size reduction

⚠ **Distroless Lesson**

- ❖ Efficiency ≠ Security
- ❖ 203 CVEs with 4 Critical

Ecosystem > Components

- ❖ Alpine shift creates different attack surface
- ❖ musl vs glibc — only 1 shared CVE

13. Limitations

Scope: Results are specific to a single, simple Python web application. Complex legacy applications may require Debian libraries (glibc) that are incompatible with Alpine.

Snapshot: Vulnerability data is based on the NVD status as of Jan 7, 2026. Continuous scanning is required as new CVEs are discovered daily.

14. Future Work

This study demonstrates Alpine's superior performance for a single, simple Python web application. Future research directions include:

- ❖ Validating findings with complex, production-grade Python applications having extensive dependencies and legacy library requirements
- ❖ Performance benchmarking between glibc (Debian) and musl (Alpine) implementations under real-world workloads
- ❖ Long-term CVE emergence tracking across different base image ecosystems

15. References

- [1] R. Shu, X. Gu, and W. Enck, "A study of security vulnerabilities on Docker Hub," in *Proc. 7th ACM Conf. Data Appl. Secur. Privacy (CODASPY)*, 2017, pp. 269–280. doi: 10.1145/3029806.3029832.
- [2] <https://trivy.dev/docs/latest/guide/>
- [3] <https://github.com/anchore/grype?tab=readme-ov-file>
- [4] <https://oss.anchore.com/docs/guides/sbom/getting-started/>
- [5] <https://github.com/wagoodman/dive>
- [6] <https://docs.docker.com/>
- [7] <https://docs.python.org/release/3.14.3/>

Contact Information:

Amit Soni Email: asoni2@uco.edu | Dr. Myungah (Grace) Park, Ph.D. Email : MPark5@uco.edu